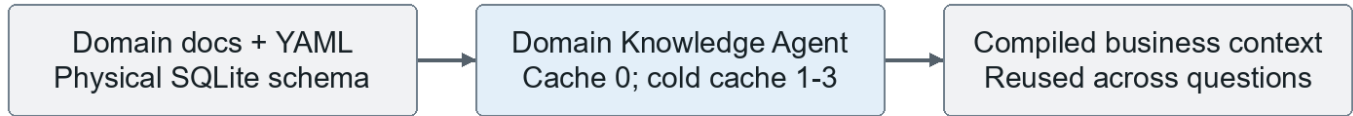


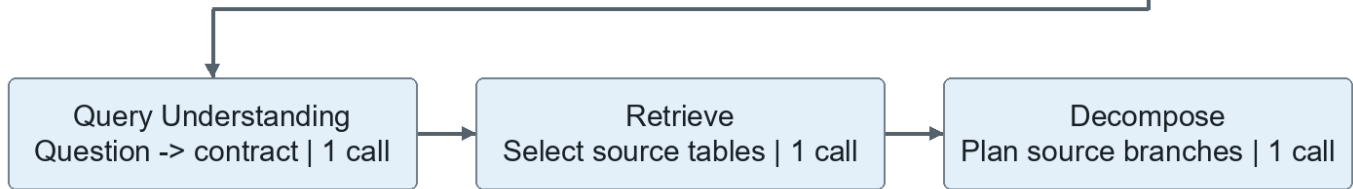
Improvement 7 Workflow Guide

Follow a question from interpretation to a saved answer. This guide explains the agents, operators, recovery limits, measured model calls and the files that control the workflow.

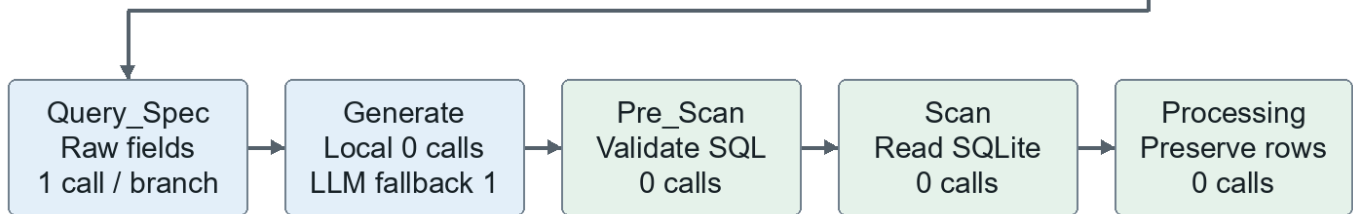
STARTUP Read-only database checks and shared business context



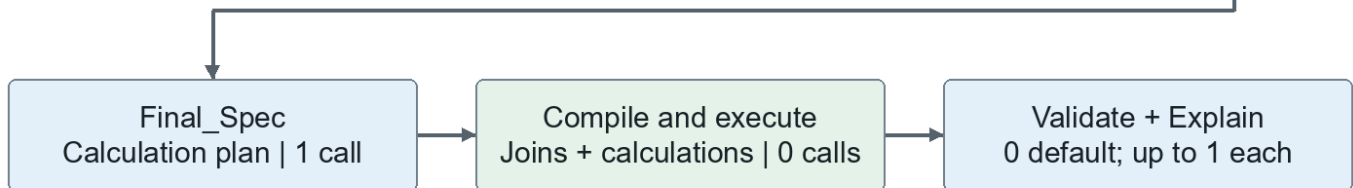
EACH QUESTION Original question + shared context enter planning



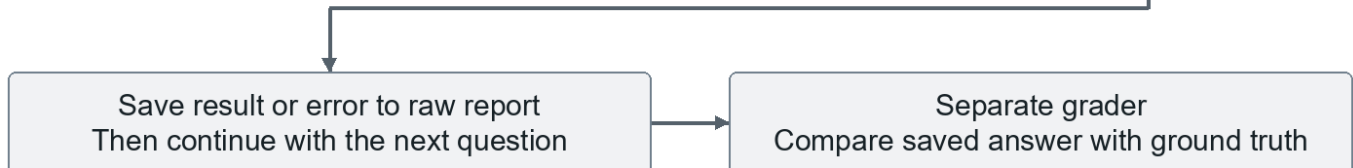
REPEAT THIS ROW FOR EACH SOURCE BRANCH B branches



ALL BRANCHES COME TOGETHER Deterministic calculation runtime



OUTPUT Raw report first; grading is a separate operation



Blue = stage that can call a model, not a guarantee that it does. Green = local code. Gray = inputs or outputs. This is the forward path; the separate recovery map shows repeated calls and consultation. Generate uses an LLM only if local rendering cannot produce SQL.

Read left to right within each row, then follow the arrow to the next row. Branches are logical data paths; the executor does not automatically run them in parallel.

What the two agents do

Domain Knowledge Agent learns the shared rules

At startup, this agent reads the authorized domain documents and schema. It produces a reusable, cited business knowledge pack. A matching cached pack avoids another model call. It does not read the test answer or query database rows.

Input	What it contributes
domain_intro_latest.prompt	General wealth management concepts and domain context.
business_rules_addendum.md	Business definitions, formulas and rules to follow.
table_metadata/*.yaml	Table and column meanings, synonyms and enum descriptions.
SQLite physical schema	Which tables and columns actually exist.

The YAML files describe the data; they are not an extra agent and do not execute rules by themselves. The physical database remains the authority for real table and column names.

Query Understanding Agent interprets one question

For each question, it uses the annotated schema and business knowledge to propose a contract: which fields mean what, needed metrics, filters, grouping, projected columns, ordering and limits. Later stages can use this contract to stay aligned with the question.

A simple illustrative question

Example: "Show total transaction amount per client for last month." This is illustrative, not a benchmark answer or a promise about exact column names.

1. Domain context supplies definitions such as what a client, amount and transaction date mean. Query Understanding binds those meanings to actual schema fields and identifies the time filter and per-client grouping.
2. Retrieve selects the relevant tables. Decompose creates the source branches. Query_Spec requests raw IDs, amounts and dates; Generate and Scan fetch the rows.
3. Final_Spec plans any needed join, filter and sum. Local operators execute that plan. Validate and Explain prepare the result for the report.

What happens when another stage asks an agent

Most stages receive the already compiled context; receiving context is not another agent call. Optional live consultation is different: with agent consultation enabled, Decompose, Query_Spec and Final_Spec can request focused advice. Generate and Scan do not directly request that advice. See the recovery page for the budget and restart behavior.

Important operators

An operator is one named step in the pipeline. A model-assisted operator proposes a plan; local Python and SQLite perform the actual data work.

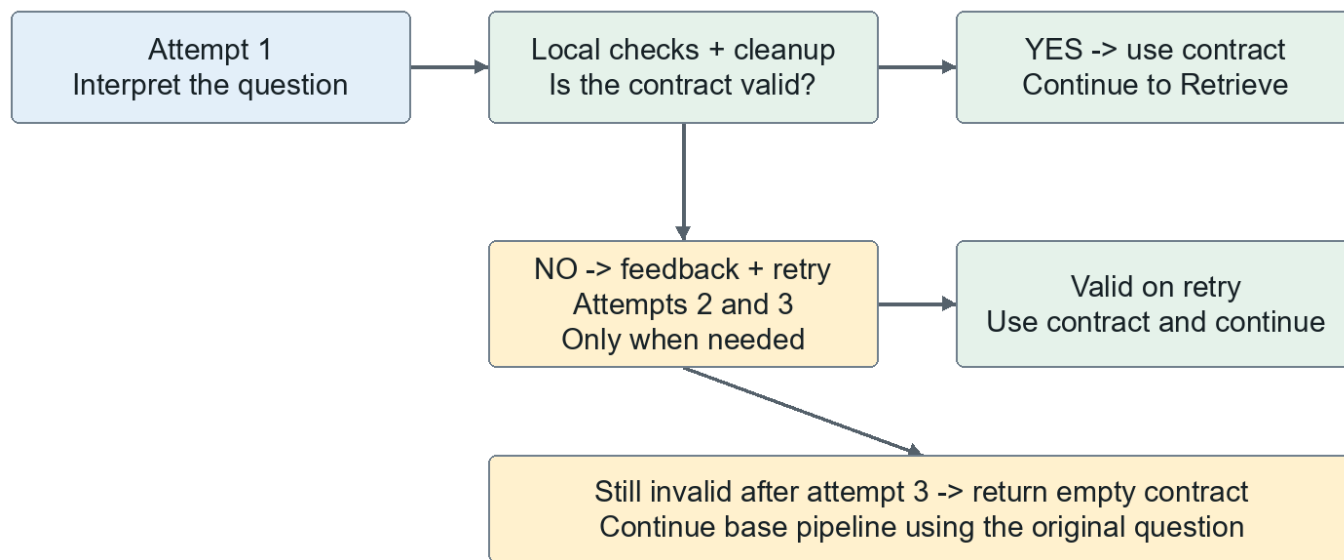
Operator	Job	Normal model calls
Retrieve	Select source tables from schema.	1
Decompose	Split the task into source branches.	1
Query_Spec	Choose raw fields and retrieval filters.	1 / branch
Generate	Render raw SQL locally, or use 1 LLM call if rendering returns None.	0 or 1
Pre_Scan_Validate	Check SQL, schema and raw retrieval rules.	0
Scan	Execute read-only SQLite retrieval.	0
Refine	Not on the normal path. Each invocation makes 1 LLM call.	Repair: 1
Processing_Spec	Keep branch rows and schema available.	0
Final_Spec	Build the final calculation plan as JSON.	1
Validate / Explain	Check and format the executed answer.	0 by default

Calculation operators inside the final plan

Operator family	Meaning
Integrate	Join source datasets and check join cardinality.
Filter_Aggregate	Filter and aggregate using sum, average, count or distinct count.
Math_Compute	Calculate derived values and arithmetic expressions.
Date_Extract / Bucket	Extract date parts or assign values to defined buckets.
Order_By / Distinct	Sort or limit output; remove duplicates.
Set_Intersect / Difference / Union	Combine or compare sets of rows.

These calculation operators make zero model calls in the current compiled data path. Integrate, Filter_Aggregate and Order_By also retain legacy LLM fallbacks outside that path. A successful execution can still be a wrong answer; the separate grader checks correctness.

Self healing and its limits



Query Understanding stops retrying as soon as a valid contract is available. After three invalid attempts, current code uses the base pipeline with the original question; it does not select an invalid "best" contract.

Stage	Bound and behavior
Domain compilation	Cache miss: up to 3 validation attempts. If all fail, startup stops.
Query_Spec	Up to 3 attempts per branch in a decomposition pass.
Pre-scan and Scan	Default 3 checks / attempts per loop, with SQL regeneration or Refine when applicable.
Final_Spec	Up to 4 attempts per execution; structured errors guide repair.
Decomposition	Up to 3 passes; missing source data or upstream planning errors can restart branches.

Optional specialist consultation

Default off. With `--agent-consultation on`, Decompose, Query_Spec and Final_Spec may request domain or interpretation advice. The shared budget is 5 specialist requests per question. A domain request uses one model call; a Query Understanding request can use up to three attempts. A changed interpretation restarts retrieval and decomposition within the existing pass budget.

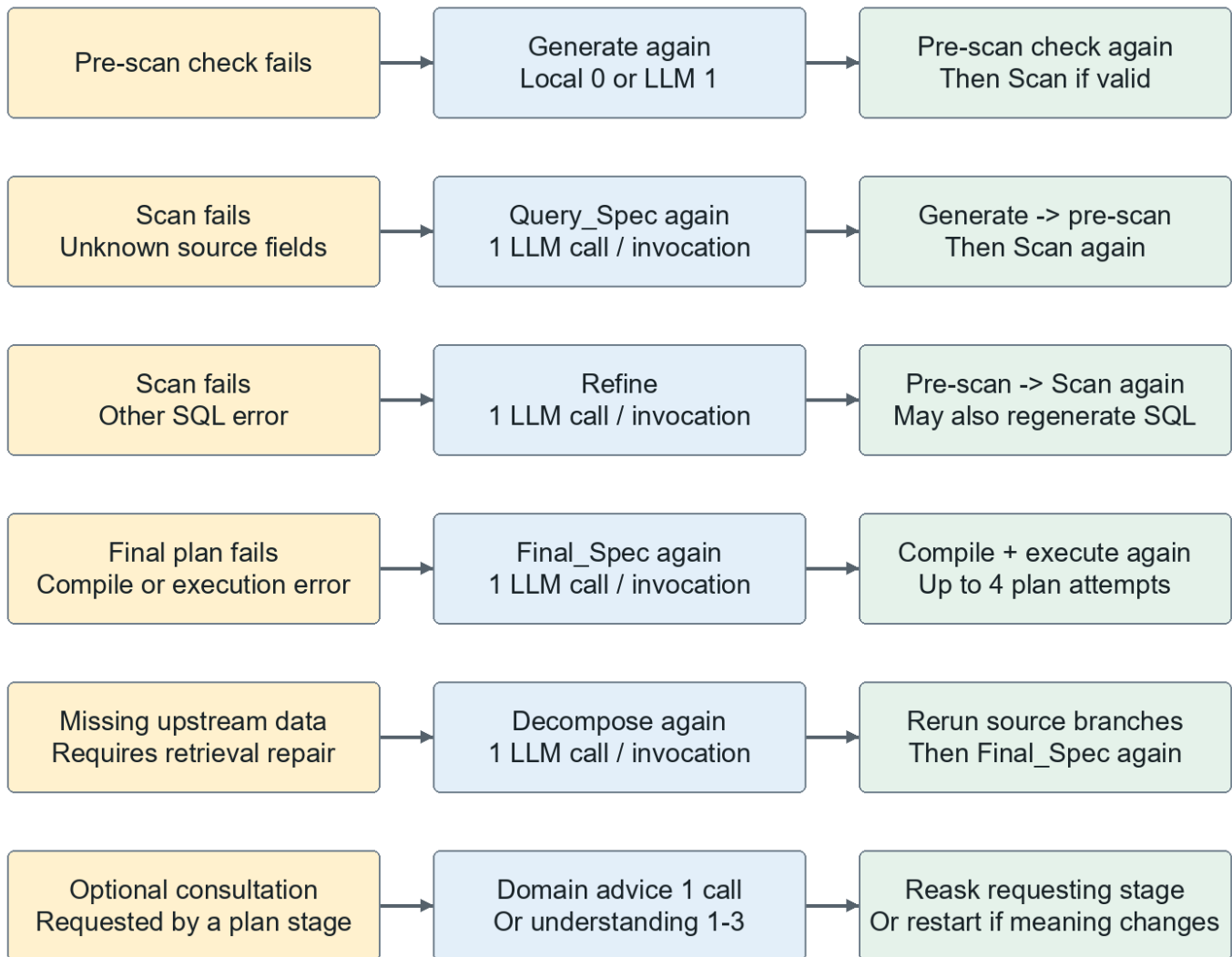
Why a question or batch can still stop

Retry limits apply to individual loops, not to the entire question. Nested retries can add many calls. Exhausted SQL or calculation repairs produce an error row and the batch normally moves on. An API quota failure can stop the batch; invalid startup knowledge, database health or report-lock problems can prevent it from starting. Transport exceptions are not guaranteed to reach the Query Understanding fallback.

Best effort means continuing where a supported fallback exists, not inventing database results or silently discarding required constraints. Valid empty results can be successful. Zero execution errors cannot be guaranteed.

All self healing paths

Each row below is a conditional recovery route, not six steps that always run. Failed attempts can add both a repair call and repeated downstream calls. Query Understanding has its own recovery diagram on the preceding page.



Pre-scan and Scan default to 3 checks or attempts per loop. Decomposition has up to 3 passes. These loops can nest, so the whole question is not limited to 3 model calls. Successful checks stop their loop early.

Generate always tries `render_raw_sql` first, including during regeneration. A self-healing event is therefore not automatically a model call. Refine, however, calls the model whenever that repair operator is invoked.

When a timeout rewrite is too similar to the failed SQL, the executor calls Generate as well. If an unknown-field Query_Spec repair is invalid, it can fall through to Refine. Missing SQL triggers Generate, pre-scan checks and another Scan.

Consultation can add the requesting operator call, the specialist call(s), and another operator call. A revised interpretation instead restarts Retrieve and Decompose. The 5-request budget is not a 5-model-call budget.

Complete model call inventory

A model-call site is not the same as an operator invocation. This inventory separates normal calls, conditional calls and compatibility code not used by the current compiled execution path.

Call site	When the model is called
Domain compilation	Cache miss: 1 call per attempt, up to 3. Cache hit: 0.
Query Understanding	1 call per attempt, up to 3 per invocation. Consultation can invoke it again.
Retrieve	1 per invocation; called again when specialist advice changes interpretation.
Decompose	1 per invocation; outer repair passes and consultation can repeat it.
Query_Spec	1 per invocation. Up to 3 branch validation attempts, plus scan repairs, restarts and consultation.
Generate	0 if render_raw_sql succeeds; otherwise 1. The same condition applies on regeneration.
Refine	1 each time the SQL repair operator is called.
Final_Spec	1 per invocation. Up to 4 plan attempts per execution, plus restarts and consultation.
Domain consultation	1 per specialist request when consultation is enabled.
Validate	1 if semantic review is enabled and local execution checks pass; otherwise 0.
Explain	1 when neither the empty/context bypass nor structured JSON path applies.
Legacy calculation fallbacks	Integrate, Filter_Aggregate and Order_By contain model fallbacks; current compiled data execution bypasses them.

Current compiled execution supplies strict_spec=True and actual datasets. Integrate returns via local relational code; Filter_Aggregate and Order_By use their data branches. Their model fallbacks are real source-code sites, but are not additional calls in this workflow.

performance.py wraps and records SDK calls; it is not an extra independent reasoning step. SDK-internal transport retries may make more network attempts than the recorded invocation count. Grading runs separately and adds its own calls.

Source audit: llm_operators/*.py; knowledge.py; query_understanding.py; consultation.py; executor.py; execution.py; main.py; performance.py. The first-page counts describe a successful forward pass, not the full repair budget.

Model calls and measured averages

Normal path formula

With Query Understanding on, cached domain knowledge, local SQL rendering, consultation off, and default validation and explanation:

Calls per question = 1 Query Understanding + 1 Retrieve + 1 Decompose + B Query_Spec + 1 Final_Spec = B + 4

B is the number of source branches, not necessarily the number of unique tables. One branch needs 5 calls; two need 6; four need 8. These are normal-path counts, not hard caps.

Add calls for failed attempts, live consultation, model SQL fallback, optional semantic review or model explanation. A successful cold-cache domain compilation usually adds one shared startup call; it can try up to three times. Turning Query Understanding off removes its normal one call.

Original eight full runs

Measured from the eight completed timing logs started at 13:46 on 3 October 2026. Every row includes 125 original question attempts, including failed questions. Later reruns and grading are excluded. These are historical measurements, not a new run of the current code.

Question type	Questions	Calls	Calls / question	Seconds / question
benchmark	125	876	7.01	48.8
at risk critical	125	1167	9.34	80.1
batch status	125	908	7.26	51.0
enrichment context	125	972	7.78	61.9
multi step comparative	125	903	7.22	81.9
reference compliance	125	820	6.56	42.9
scoring quantitative	125	841	6.73	48.5
temporal transaction	125	776	6.21	46.4
All types	1000	7263	7.26	57.7

Across the 1,000 original attempts: 7,263 model calls, averaging 7.26 per question. Average recorded processing time was 57.7 seconds per question; this is not the total elapsed time when multiple terminals run together.

A call means one instrumented SDK invocation. Internal SDK retries are not separate entries; their waiting time can be included. Startup calls have no question ID and are excluded from this average. Grader calls are also excluded.

Audit source: improvement7/.runtime/logs/timing_20261003_1346*.json. workflow_metrics.json lists the exact eight files, totals and stage counts so this snapshot can be checked.

How to change the workflow

Edit the picture or edit the running pipeline

The Word text is editable. Its diagrams are images for stable layout. Open Improvement7_Workflow.drawio in a compatible diagram editor to edit boxes and arrows. Its three pages show the main workflow, Query Understanding recovery and all self-healing paths. Editing these documents does not change pipeline behavior.

For actual behavior changes, use the code map below. Paths are relative to improvement7/run_pipeline_v2_sql unless stated otherwise.

Change you want	File or entry point
Operator order or branches	planner.py: AdvancedAOPPlanner._build_three_spec_dag
Startup, agent wiring, outer retries and reports	main.py: _main and run_single_test
Stage dispatch, local retries and repair routing	executor.py: AOPExecutor.execute_dag
Question contract and its 3-attempt fallback	query_understanding.py: understand_query
Domain compilation, cache and citations	knowledge.py: compile_knowledge
Live advice and its shared request budget	consultation.py: ConsultationSession
Final plan prompting and supported calculations	llm_operators/final_spec.py; spec_runtime.py; spec_contracts.py; non_llm_operators/
Defaults and command-line switches	common.py and run.py
Business meaning without changing the DAG	../business_rules_addendum.md; ../domain_intro_latest.prompt; ../table_metadata/*.yaml

Make a small controlled change

1. Decide whether the change is business meaning, interpretation, operator order or execution. Change only the owning layer. Adding a new operator also requires registry wiring, compiler support, prompt updates and tests; an arrow alone is not enough.
2. Keep the test set and ground truth out of agent prompts and business rules. Define general behavior, then test it without encoding individual benchmark answers.
3. Run the existing unit tests from the financial folder:

```
python -B -m unittest discover -s improvement7/run_pipeline_v2_sql/tests -v
```

4. Use a separate output report for a full comparison after changing code. In-place error retries keep successful rows from older runs, so that report is a mixed-run result, not a clean full rerun.

This guide was generated from the local code and saved timing logs. No pipeline logic, business rules, test questions or existing results were changed for this documentation task.